Thinking in parallel

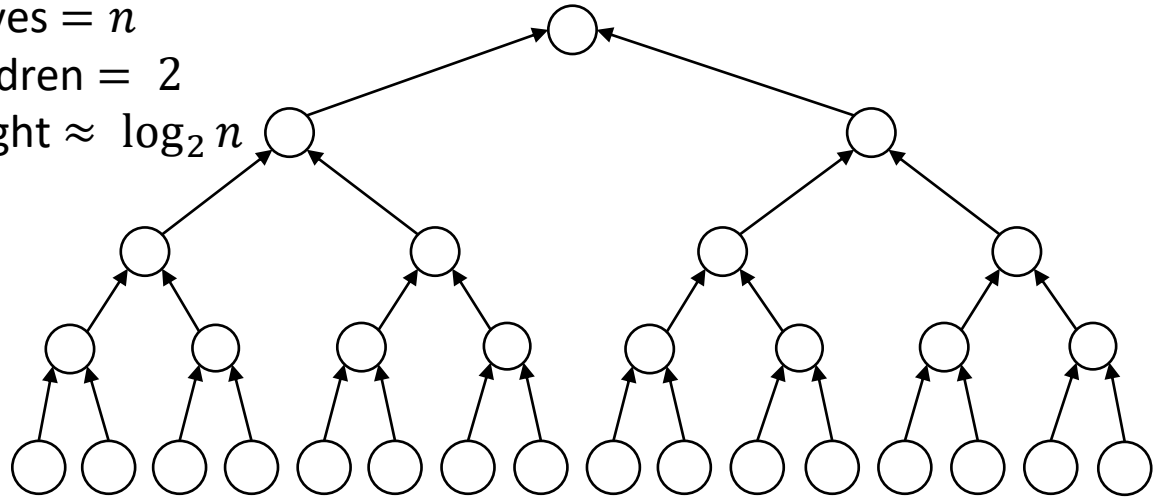
How to design algorithms for multi-processor systems

Basic techniques

Basic techniques: Doubly logarithmic-depth tree

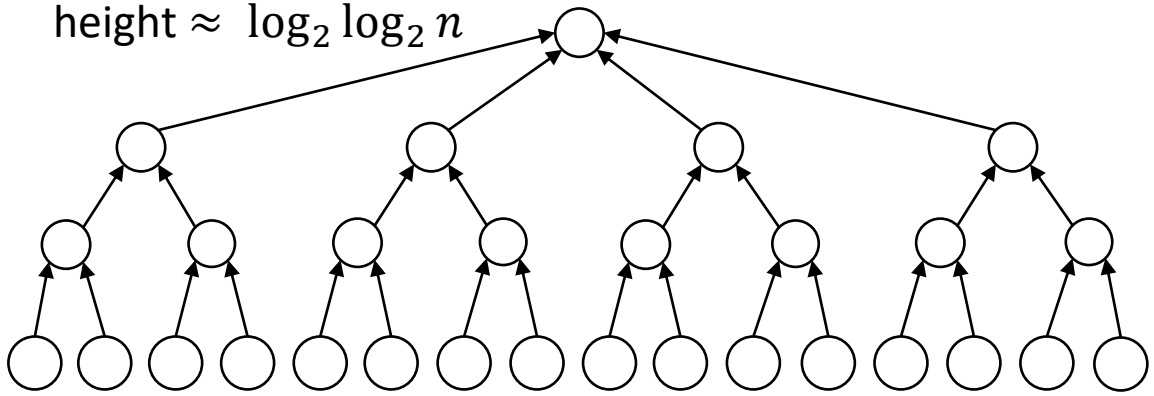
- it is based on doubly logarithmic-depth tree scheme
  - a node has as many children as the square root of the total number of leaves in its subtree
- faster than the balanced binary tree scheme
  - but we need a constant-time algorithm to solve the problem in each internal node

leaves $= n$
children $= 2$
height $\approx \log_2 n$



Balanced binary tree scheme

leaves $= n$
children $= \sqrt{leaves\ in\ subtree}$
height $\approx \log_2 \log_2 n$



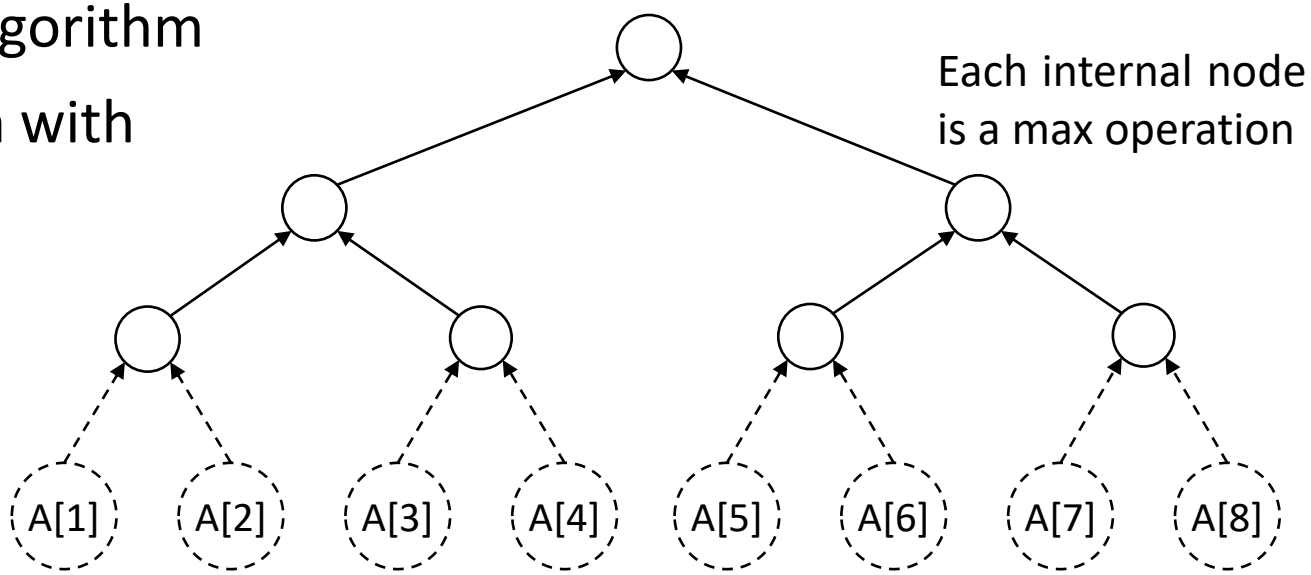
doubly logarithmic-depth tree

Doubly logarithmic-depth tree: Max of an array

MAXIMUM OF AN ARRAY
Input: An array A of n elements.
Output: $\max_{1 \le i \le n} A[i]$.

- There is a sequential $O(n)$-time algorithm
- There is an EREW PRAM algorithm with
  - time $O(\log n)$
  - work $O(n)$ (it is work optimal)

GOAL: Work optimal PRAM algorithm with better time



Balanced binary tree scheme

A constant-time algorithm

ALGORITHM Basic_Maximum

/* Compare all pairs of elements */

1. **for** $1 \leq i, j \leq n$ **pardo**
2. **if** $A[i] \geq A[j]$ **then** $B[i,j] \leftarrow 1$ **else** $B[i,j] \leftarrow 0$

/* Compute the maximum element in array M. M[i]=1 iff A[i] is the maximum */

/* A[i] is the maximum iff B[i,j]=1 for every j /*

3. **for** $1 \leq i \leq n$ **pardo**
4. $M[i] \leftarrow B[i,1]$ **and** $B[i,2]$ **and** … **and** $B[i,n]$
5. **for** $1 \leq i \leq n$ **pardo if** $M[i]=1$ **then output** $A[i]$

ISSUE: How do we implement step 4 in parallel constant time on a common CRCW?

A constant-time algorithm

ALGORITHM Basic_Maximum

/* Compare all pairs of elements */

1. **for** $1 \leq i, j \leq n$ **pardo**
2. **if** $A[i] \geq A[j]$ **then** $B[i,j] \leftarrow 1$ **else** $B[i,j] \leftarrow 0$

/* Compute the maximum element in array M. M[i]=1 iff A[i] is the maximum */

/* A[i] is NOT the maximum iff B[i,j]=0 for SOME j /*

3. **for** $1 \leq i \leq n$ **pardo** $M[i] \leftarrow 1$
4. **for** $1 \leq i, j \leq n$ **pardo**
5. **if** $B[i,j] = 0$ **then** $M[i] \leftarrow 0$
6. **for** $1 \leq i \leq n$ **pardo**
7. **if** $M[i]=1$ **then output** $A[i]$

- Time $O(1)$ (optimal)
- Work $O(n^2)$ (not optimal)
- <u>common</u> CRCW PRAM algorithm
  - Concurrent write same variable in step 5
  - but they all write the same value 0 on $M[i]$

A constant-time algorithm

THEOREM: Given an array A of n elements, we can compute the maximum element of A on a common CRCW PRAM in time $O(1)$ and work $O(n^2)$.

Doubly logarithmic-depth tree

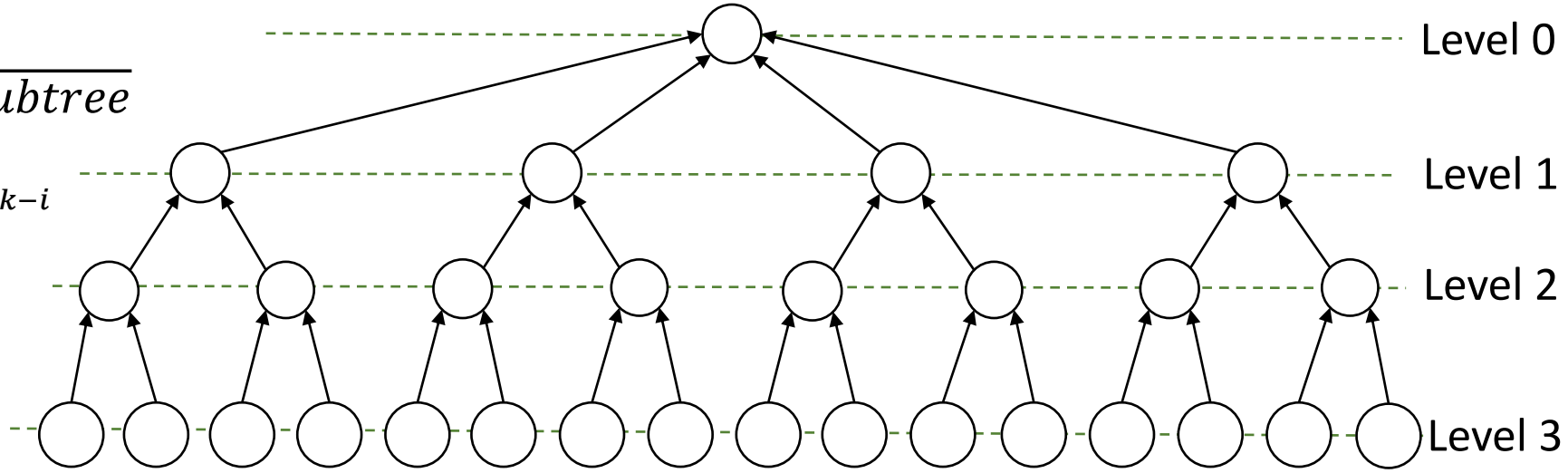
DEFINITION: The doubly logarithmic-depth tree with $n = 2^{2^k}$ leaves is a rooted tree defined recursively. For $k = 0$, the root has two children that are also leaves. For $k \geq 1$, the root has $2^{2^{k-1}}$ children, each of which is a doubly logarithmic-depth tree with $2^{2^k - 2^{k-1}} = 2^{2^{k-1}}$ leaves.

leaves: $n = 2^{2^k}$

children: $\sqrt{leaves\ in\ subtree}$

height $= k + 1$

#nodes in level i: $2^{2^k - 2^{k-i}}$



doubly logarithmic-depth tree

Properties

LEMMA: Let T be a doubly logarithmic-depth tree with $n = 2^{2^k}$ leaves. The number of nodes of T at level $i < k$ is equal to $2^{2^k - 2^{k-i}}$. The number of nodes at level k is $2^{2^k - 1}$. The height of T is $k + 1$.

PROOF (only correctness): By induction on i.

BASE CASE $i = 0$: $2^{2^k - 2^k} = 2^0 = 1$. Level 0 contains the root of T only.

INDUCTIVE CASE $i + 1$: By induction, there are $2^{2^k - 2^{k-i}}$ nodes at level i. Each node at level i has $2^{2^{k-i-1}}$ children. Therefore, the number of nodes at level $i + 1$ is

$$2^{2^{k-i-1}} \cdot 2^{2^k - 2^{k-i}} = 2^{2^k - 2^{k-i} + 2^{k-i-1}} = 2^{2^k - 2^{k-i-1} \cdot (2-1)} = 2^{2^{k-(i+1)}}.$$

#NODES AT LEVEL k: There are $2^{2^k - 2^{k-(k-1)}} = 2^{2^k - 2} =$ nodes in level $k - 1$, each of which has 2 children. Therefore, the number of nodes at level k is $2 \cdot 2^{2^k - 2} = 2^{2^k - 1}$.

HEIGHT OF THE TREE: there are $2^{2^k - 1}$ nodes at level k, each of which has 2 children. Therefore, the number of nodes at level $k + 1$ is $2 \cdot 2^{2^k - 1} = 2^{2^k} = n$. Thus, all the leaves of T are in level $k + 1$.

An algorithm with better work

ALGORITHM Doubly_Logarithmic_Maximum

/* Assume that n=2^{2^k}, where k is an integer. Elements of A are in the leaves */

1. **for** i = k **downto** 0
2. **for every** node x of level i of the doubly logarithmic-depth tree T **pardo**
3. compute max among elements in the children of x using Basic_Maximum

- Time $O(\log\log n)$ (better than $O(\log n)$)
- Work $O(n\log\log n)$ (not optimal)
- <u>common</u> CRCW PRAM algorithm

The total work in level k is $O(n)$ as each node has constant degree.
The total work in level $i < k$ is number of nodes in the level times number of children squared, i.e.,

$$O\left(2^{2^k-2^{k-i}}\cdot\left(2^{2^{k-i-1}}\right)^2\right)=O\left(2^{2^k}\right)=O(n)$$

Doubly logarithmic-depth tree: Max of an array

THEOREM: Given an array A of n elements, we can compute the maximum element of A on a common CRCW PRAM in time $O(\log\log n)$ and work $O(n\log\log n)$.

Basic techniques: Accelerated cascading

- Useful for designing a performant algorithm given
  - ALGORITHM 1: makes little work but has a slow run time
  - ALGORITHM 2: fast run time but makes lot of work
- Allows us to take (almost) the best of both algorithms
  - **PHASE 1**: use ALGORITHM 1 until the size of the problem is below a certain threshold value
  - **PHASE 2**: Then, shift to ALGORITHM 2

	ALGORITHM 1		ALGORITHM 2
TIME	SLOW	>	FAST
WORK	FAST	<	SLOW

ACCELERATED CASCADING

	FAST ALGORITHM
TIME	FAST
WORK	FAST

Accelerated cascading

ALGORITHM Optimal_Maximum

/* Phase 1 */

1. Use the balanced binary tree scheme and move up for $\approx \log\log n$ levels

Time $O(\log\log n)$
Work $O(n)$

/* Phase 2. We are left with O(n/log n) elements */

2. Continue using the doubly logarithmic-depth tree scheme

Time $O(\log\log n)$
Work $O\left(\frac{n}{\log n}\log\log n\right) = O(n)$

	BALANCED BINARY TREE		DOUBLY LOGARITHMIC-DEPTH TREE
TIME	$O(\log n)$ $O(1)$ per level	>	$O(\log\log n)$ $O(1)$ per level
WORK	$O(n)$	<	$O(n\log\log n)$

ACCELERATED CASCADING

	FAST ALGORITHM
TIME	$O(\log\log n)$
WORK	$O(n)$

- Time $O(\log\log n)$
- Work $O(n)$ (optimal)
- <u>common</u> CRCW PRAM algorithm

Accelerated cascading from another perspective

ALGORITHM Optimal_Maximum_Bis

/* Phase 1 */

1. Divide A into n/log n subarrays of size log n each
2. Solve the problem on each subarray concurrently using the balanced binary tree technique

/* Phase 2. We are left with O(n/log n) elements */

3. Continue using the doubly logarithmic-depth tree scheme

- Time $O(\log\log n)$
- Work $O(n)$ (optimal)
- <u>common</u> CRCW PRAM algorithm

Time $O(\log\log n)$
Work $O(n)$

Time $O(\log\log n)$
Work $O\left(\frac{n}{\log n}\log\log n\right) = O(n)$

	BALANCED BINARY TREE		DOUBLY LOGARITHMIC-DEPTH TREE
TIME	$O(\log n)$ $O(1)$ per level	>	$O(\log\log n)$ $O(1)$ per level
WORK	$O(n)$	<	$O(n\log\log n)$

ACCELERATED CASCADING

	FAST ALGORITHM
TIME	$O(\log\log n)$
WORK	$O(n)$

Accelerated cascading: Max of an array

THEOREM: Given an array A of n elements, we can compute the maximum element of A on a common CRCW PRAM in time $O(\log\log n)$ and work $O(n)$.

THEOREM (without proof): Given an array A of n elements, any CRCW PRAM algorithm with $O(n)$ work needs $\Omega(\log\log n)$ time to compute the maximum element of A, regardless of whether it is common, arbitrary, or priority.

OBSERVATION: We designed a work-time optimal CRCW PRAM algorithm

THEOREM (without proof): Given an array A of n elements, any CREW PRAM algorithm requires $\Omega(\log n)$ time to compute the maximum element of A.

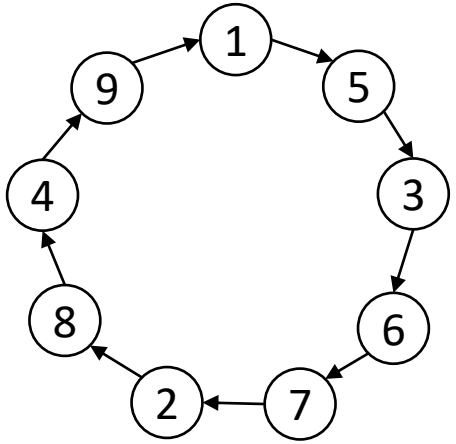
OBSERVATION: The balanced binary tree scheme yields a work-time optimal EREW PRAM algorithm

Basic techniques: Symmetry breaking

- Useful when symmetry seems an unsurpassable barrier for concurrency

Symmetry breaking: 3-coloring a cycle

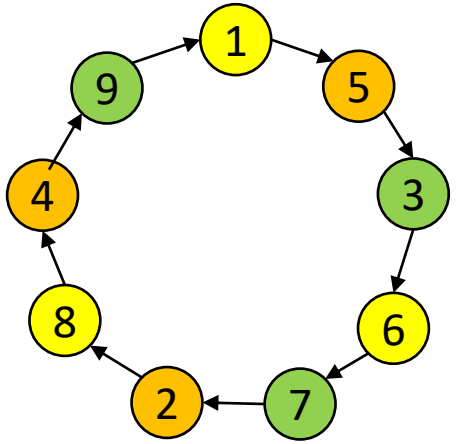
3-COLORING A CYCLE
Input: A directed cycle of n labelled nodes, represented by an array S, where S[i] represents the successor of node (with label) i.
Output: A 3-coloring C of the nodes, where $C[i] \neq C[S[i]]$, for every i.



index	S
1	5
2	8
3	6
4	9
5	3
6	7
7	2
8	4
9	1

Symmetry breaking: 3-coloring a cycle

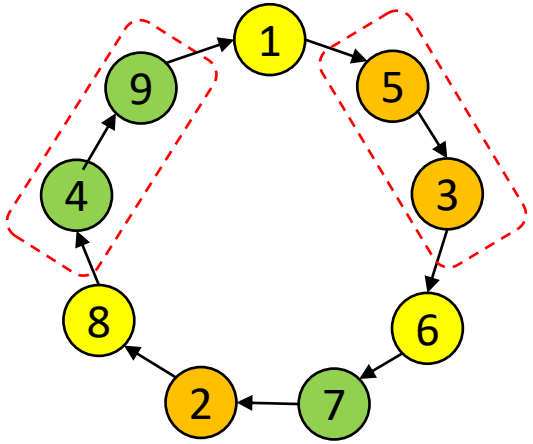
3-COLORING A CYCLE
Input: A directed cycle of n labelled nodes, represented by an array S, where S[i] represents the successor of node (with label) i.
Output: A 3-coloring C of the nodes, where $C[i] \neq C[S[i]]$, for every i.



A valid 3-coloring

Symmetry breaking: 3-coloring a cycle

3-COLORING A CYCLE
Input: A directed cycle of n labelled nodes, represented by an array S, where S[i] represents the successor of node (with label) i.
Output: A 3-coloring C of the nodes, where $C[i] \neq C[S[i]]$, for every i.



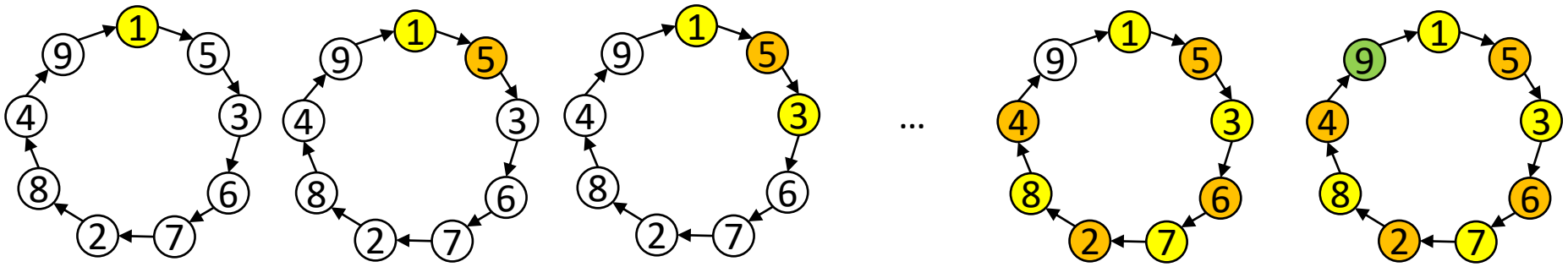
Not a valid 3-coloring

Symmetry breaking: 3-coloring a cycle

3-COLORING A CYCLE
Input: A directed cycle of n labelled nodes, represented by an array S, where S[i] represents the successor of node (with label) i.
Output: A 3-coloring C of the nodes, where $C[i] \neq C[S[i]]$, for every i.

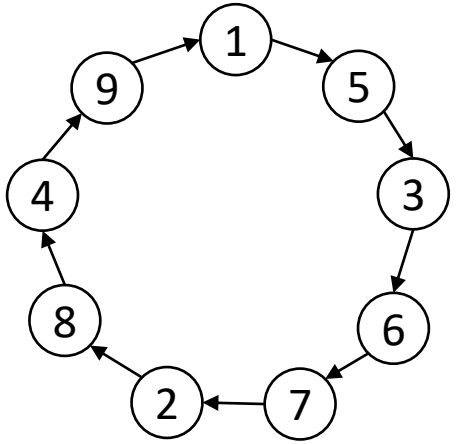
- $O(n)$-time sequential algorithm with at most 3 colors
  - Start from any node and try to color all the cycle with 2 alternating colors
  - Need to use an extra color on the last uncolored node iff the cycle is odd

Not parallelizable…
…due to symmetries



A simple solution with n colors

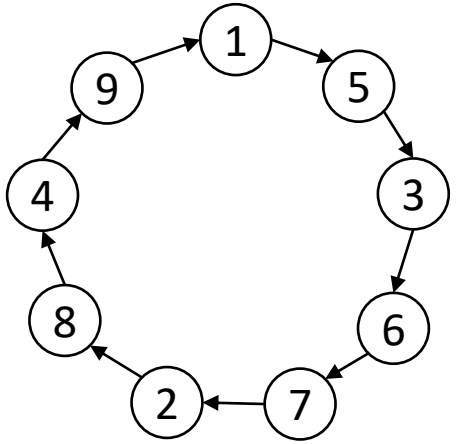
- We assume to also have the array P such that $P[i]$ is the predecessor of node i
  - **for** 1 ≤ i ≤ n **pardo** P[S[i]] ← i
- Can we color the cycle using more than 3 colors (or numbers)?
  - YES with n colors! Color each node using its label (we already broke symmetries)



index	S	P	C
1	5	9	1
2	8	7	2
3	6	5	3
4	9	8	4
5	3	1	5
6	7	3	6
7	2	6	7
8	4	2	8
9	1	4	9

How to reduce the number of colors

- Can we reduce the number of colors and still have a valid coloring?
  - YES!
  - Consider the binary representation of colors
  - Denote by $C[i]_k$ the k-th bit of the binary representation of $C[i]$
  - If k is the least significant bit position for which $C[i]_k \neq C[S[i]]_k$ then
    - $C'[i] \leftarrow 2(k-1) + C[i]_k$



index	S	P	C	C'
1	5	9	0001	4
2	8	7	0010	3
3	6	5	0011	1
4	9	8	0100	0
5	3	1	0101	2
6	7	3	0110	0
7	2	6	0111	1
8	4	2	1000	4
9	1	4	1001	7

Basic coloring algorithm

ALGORITHM Basic_Coloring

/* Input: A directed cycle of n labeled nodes modeled with an array S of successors, and a coloring C */

/* Output: a new coloring C */

1. **for** $1 \leq i \leq n$ **pardo**
2. $k \leftarrow$ the least significant bit position in which $C[i]_k \neq C[S[i]]_k$
3. $C[i] \leftarrow 2(k-1) + C[i]_k$

- We assume that k can be computed in $O(1)$ time
  - it heavily depends on the type of primitive operations supported by our processors

- Time $O(1)$ (optimal)
- Work $O(n)$ (optimal)
- CREW PRAM algorithm

Basic coloring algorithm: proof of correctness

LEMMA: The Basic_Coloring algorithm computes a valid coloring in time $O(1)$ and work $O(n)$ on a EREW PRAM. If the largest used color in the initial input coloring is at least 6 (uses $t \geq 3$ bits), then the new coloring uses at most $2t$ colors from 0 to $2t - 1$, and each color needs at most $\lceil \log_2 t \rceil + 1$ bits.

PROOF (only correctness):

VALID COLORING: Let i be any node and let $j = S[i]$. As the input coloring C is a valid coloring, $C[i] \neq C[j]$. Therefore, $k \leq t$ always exists. Let $h \leq t$ be the least bit position in which $C[j] \neq C[S[j]]$. The new colors assigned to i and j are, respectively, $2(k-1) + C[i]_k$ and $2(h-1) + C[j]_h$.

We show that the two new colors assigned to i and j differ. For the sake of contradiction, assume that

$$2(k-1) + C[i]_k = 2(h-1) + C[j]_h$$

As each second term is a bit, this can only happen when $2(k-1) = 2(h-1)$, i.e., $h = k$. But then $C[i]_k = C[j]_h = C[j]_k$, thus contradicting the fact that $C[i]_k \neq C[j]_k$.

NUMBER OF BITS: the smallest color is 0. The largest color is $2(t-1) + 1 = 2t - 1$. So, in total we have 2t colors, that can be represented with $\lceil \log_2 2t \rceil = \lceil \log_2 2 + \log_2 t \rceil = \lceil \log_2 t \rceil + 1$ bits.

Check your understanding

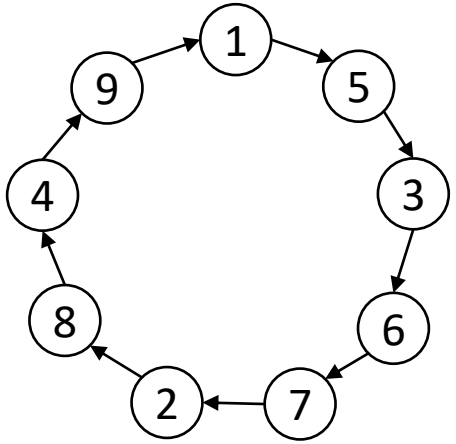
Can you modify the Basic_Coloring algorithm to let it run on an EREW PRAM in time $O(1)$ and work $O(n)$?

Recursive use of the algorithm

- Can we keep reducing the number of colors and still have a valid coloring?
  - YES!
  - Keep using the Basic_Coloring EREW PRAM algorithm as long as the largest color is at least 6
  - $n \to\approx \log_2 n \to\approx \log_2 \log_2 n \to\approx \log_2 \log_2 \log_2 n \to \cdots \to \log^* n$
  - After $O(\log^* n)$ iterations we obtain a valid coloring using at most 6 colors from 0 to 5
  - We stop when the largest color is at most 5 (cannot apply the lemma)

DEFINITION: $\log^{(i)} n = \begin{cases} \log_2 n & \text{if } i = 1 \\ \log_2\left(\log^{(i-1)} n\right) & \text{if } i > 1 \end{cases}$

DEFINITION: $\log^* n = \min\{i : \log^{(i)} n \leq 1\}$.



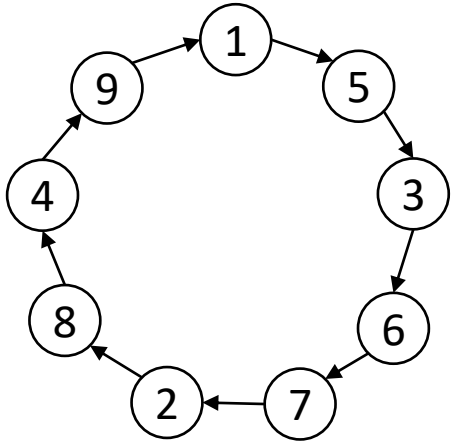
index	S	P	C	C'
1	5	9	4	
2	8	7	3	
3	6	5	1	
4	9	8	0	
5	3	1	2	
6	7	3	0	
7	2	6	1	
8	4	2	4	
9	1	4	7	

Recursive use of the algorithm

- Can we keep reducing the number of colors and still have a valid coloring?
  - YES!
  - Keep using the Basic_Coloring EREW PRAM algorithm as long as the largest color is at least 6
  - $n \to\approx \log_2 n \to\approx \log_2 \log_2 n \to\approx \log_2 \log_2 \log_2 n \to \cdots \to \log^* n$
  - After $O(\log^* n)$ iterations we obtain a valid coloring using at most 6 colors from 0 to 5
  - We stop when the largest color is at most 5 (cannot apply the lemma)

DEFINITION: $\log^{(i)} n = \begin{cases} \log_2 n & \text{if } i = 1 \\ \log_2\left(\log^{(i-1)} n\right) & \text{if } i > 1 \end{cases}$

DEFINITION: $\log^* n = \min\{i : \log^{(i)} n \leq 1\}$.



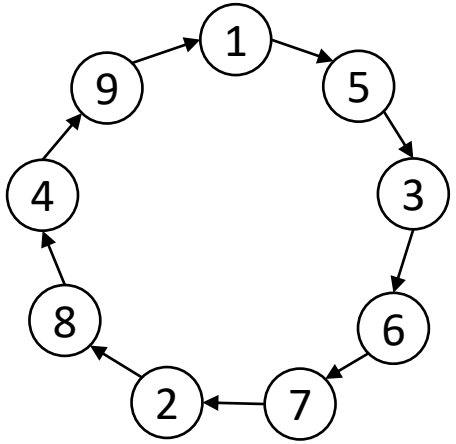
index	S	P	C	C'
1	5	9	100	2
2	8	7	011	1
3	6	5	001	1
4	9	8	000	0
5	3	1	010	0
6	7	3	000	0
7	2	6	001	2
8	4	2	100	4
9	1	4	111	1

Recursive use of the algorithm

- Can we keep reducing the number of colors and still have a valid coloring?
  - YES!
  - Keep using the Basic_Coloring EREW PRAM algorithm as long as the largest color is at least 6
  - $n \to\approx \log_2 n \to\approx \log_2 \log_2 n \to\approx \log_2 \log_2 \log_2 n \to \cdots \to \log^* n$
  - After $O(\log^* n)$ iterations we obtain a valid coloring using at most 6 colors from 0 to 5
  - We stop when the largest color is at most 5 (cannot apply the lemma)

DEFINITION: $\log^{(i)} n = \begin{cases} \log_2 n & \text{if } i = 1 \\ \log_2\left(\log^{(i-1)} n\right) & \text{if } i > 1 \end{cases}$

DEFINITION: $\log^* n = \min\{i : \log^{(i)} n \leq 1\}$.



index	S	P	C
1	5	9	2
2	8	7	1
3	6	5	1
4	9	8	0
5	3	1	0
6	7	3	0
7	2	6	2
8	4	2	4
9	1	4	1

Reducing the number of colors to ≤ 3

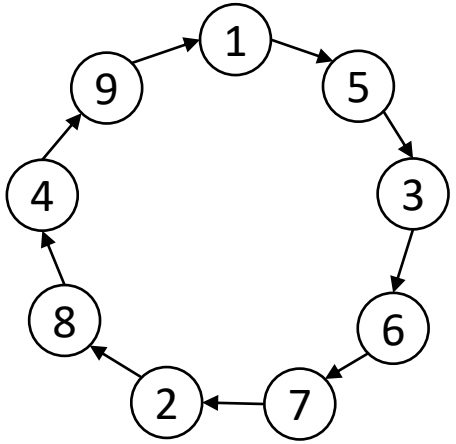
- Can we reduce the number of colors to 3?
  - YES, in (optimal) time $O(1)$ and (optimal) work $O(n)$ on an EREW PRAM

for every color c **from** 3 **to** 5 **do**

  for $1 \leq$ i $\leq$ n **pardo**

    if C[i] = c **then** recolor node i with a color in {0,1,2} that is different from C[S[i]] and C[P[i]]

- C' removes 3
- C'' removes 4
- C''' removes 5



index	S	P	C	C'	C''	C'''
1	5	9	2	2	2	2
2	8	7	1	1	1	1
3	6	5	1	1	1	1
4	9	8	0	0	0	0
5	3	1	0	0	0	0
6	7	3	0	0	0	0
7	2	6	2	2	2	2
8	4	2	4	4	2	2
9	1	4	1	1	1	1

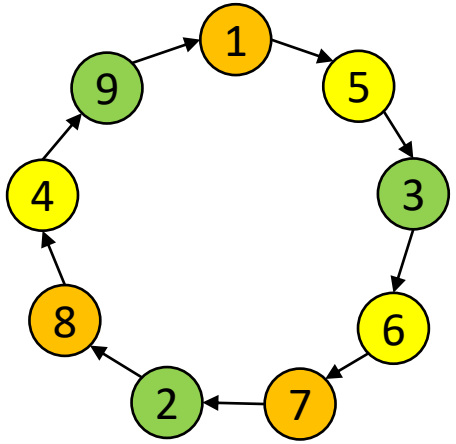
Reducing the number of colors to ≤ 3

- Can we reduce the number of colors to 3?
  - YES, in (optimal) time $O(1)$ and (optimal) work $O(n)$ on an EREW PRAM

for every color c **from** 3 **to** 5 **do**

  for $1 \leq$ i $\leq$ n **pardo**

    if C[i] = c **then** recolor node i with a color in {0,1,2} that is different from C[S[i]] and C[P[i]]



index	S	P	C
1	5	9	2
2	8	7	1
3	6	5	1
4	9	8	0
5	3	1	0
6	7	3	0
7	2	6	2
8	4	2	2
9	1	4	1

The main theorem

THEOREM: We can compute a 3-coloring of a directed cycle of n labelled nodes on an EREW PRAM with n nodes in time $O(\log^* n)$ and work $O(n \log^* n)$.

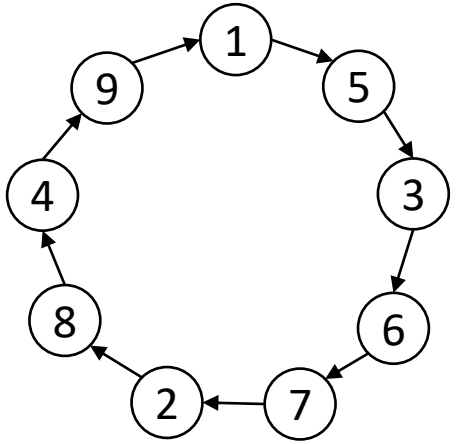
A work-optimal 3-coloring algorithm

THEOREM (SORTING SMALL INTEGERS): An array of n integers in the range $[0, O(\log n)]$ can be sorted on an EREW PRAM in time $O(\log n)$ and work $O(n)$.

SKETCH OF PROOF: It uses a non-trivial combination of the well-known radix-sort algorithm and the prefix-sum algorithm. The details are omitted as they do not fit the scope of this course.

A work-optimal 3-coloring algorithm

- Run the Basic_Coloring algorithm to reduce the number of colors to $O(\log n)$
- Sort the array by colors
- For every used color $c = 3$ to $O(\log n)$
  - Concurrently recolor all nodes with color c by assigning a valid color in $\{0,1,2\}$



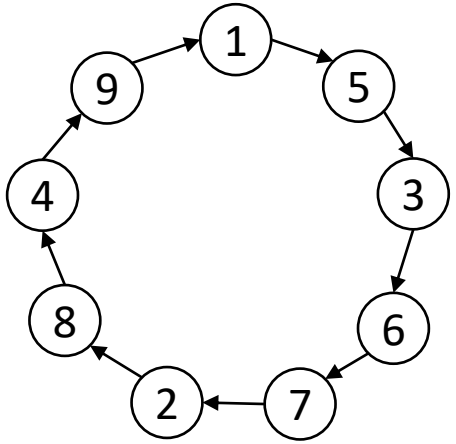
index	S	P	C	C'
1	5	9	0001	4
2	8	7	0010	3
3	6	5	0011	1
4	9	8	0100	0
5	3	1	0101	2
6	7	3	0110	0
7	2	6	0111	1
8	4	2	1000	4
9	1	4	1001	7

A work-optimal 3-coloring algorithm

- Run the Basic_Coloring algorithm to reduce the number of colors to $O(\log n)$
- Sort the array by color
- For every used color $c = 3$ to $O(\log n)$
  - Concurrently recolor all nodes with color c by assigning a valid color in $\{0,1,2\}$

index	S	P	C	C'
1	5	9	4	4
2	8	7	3	0
3	6	5	1	1
4	9	8	0	0
5	3	1	2	2
6	7	3	0	0
7	2	6	1	1
8	4	2	4	4
9	1	4	7	7

$c = 3$

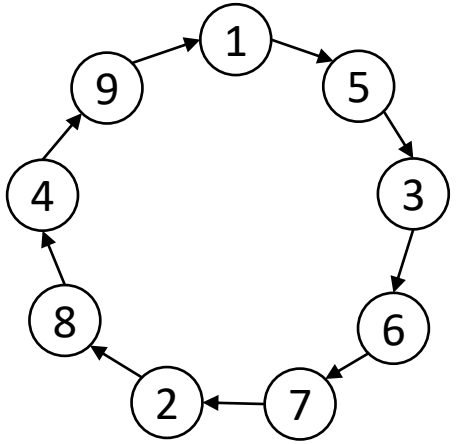


A work-optimal 3-coloring algorithm

- Run the Basic_Coloring algorithm to reduce the number of colors to $O(\log n)$
- Sort the array by color
- For every used color $c = 3$ to $O(\log n)$
  - Concurrently recolor all nodes with color c by assigning a valid color in $\{0,1,2\}$

index	S	P	C	C'
1	5	9	4	0
2	8	7	0	0
3	6	5	1	1
4	9	8	0	0
5	3	1	2	2
6	7	3	0	0
7	2	6	1	1
8	4	2	4	1
9	1	4	7	7

$c = 4$

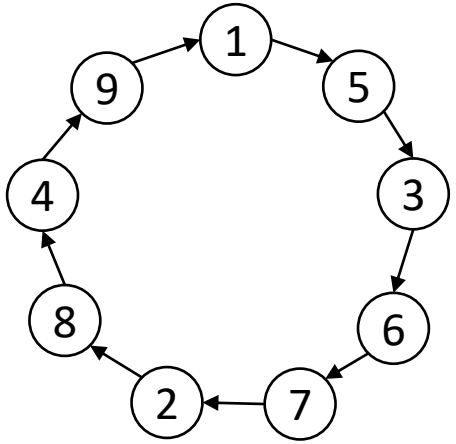


A work-optimal 3-coloring algorithm

- Run the Basic_Coloring algorithm to reduce the number of colors to $O(\log n)$
- Sort the array by color
- For every used color $c = 3$ to $O(\log n)$
  - Concurrently recolor all nodes with color c by assigning a valid color in $\{0,1,2\}$

index	S	P	C	C'
1	5	9	0	0
2	8	7	0	0
3	6	5	1	1
4	9	8	0	0
5	3	1	2	2
6	7	3	0	0
7	2	6	1	1
8	4	2	1	1
9	1	4	7	1

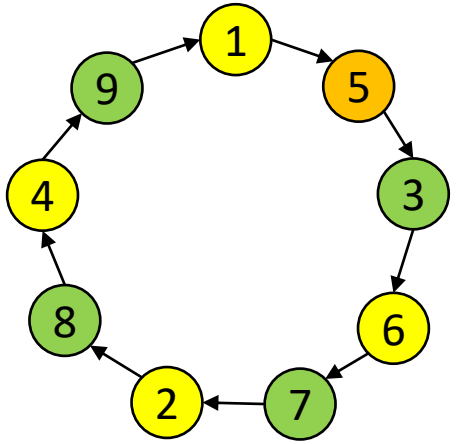
$c = 7$



A work-optimal 3-coloring algorithm

- Run the Basic_Coloring algorithm to reduce the number of colors to $O(\log n)$
- Sort the array by color
- For every used color $c = 3$ to $O(\log n)$
  - Concurrently recolor all nodes with color c by assigning a valid color in $\{0,1,2\}$

index	S	P	C
1	5	9	0
2	8	7	0
3	6	5	1
4	9	8	0
5	3	1	2
6	7	3	0
7	2	6	1
8	4	2	1
9	1	4	1



A work-optimal 3-coloring algorithm

ALGORITHM Optimal_Coloring

/* Input: A directed cycle of n labeled nodes modeled with an array S of successors, and a coloring C */

/* Output: a new coloring C */

1. **for** $1 \leq i \leq n$ **pardo** P[S[i]] ← i

2. **for** $1 \leq i \leq n$ **pardo**

3. C[i] ← i

4. Run the algorithm Basic_Coloring once to reduce the number of colors to $\lceil \log_2 n \rceil + 1$

5. Sort the array by color

6. **for** c = 3 **to** $\lceil \log_2 n \rceil + 1$

7. **for** all vertices i of color c **pardo**

8. recolor i with a color in {0,1,2} that is different from C[S[i]] and C[P[i]]

- Time $O(\log n)$ and work $O(n)$ (work optimal)

A work optimal 3-coloring algorithm

THEOREM: We can compute a 3-coloring of a directed cycle of n labelled nodes on a EREW PRAM with n nodes in time $O(\log n)$ and work $O(n)$.

PROOF:
CORRECTNESS: the only unclear part is how to perform step 8 concurrently in time $O(1)$.
CHECK YOUR UNDERSTANDING: Design an EREW PRAM algorithm that, given the nodes sorted by colors, computes the location of the first and the last nodes of each color in $O(1)$ time and $O(n)$ work. Start by designing a CREW PRAM algorithm, then try to adjust it to run on an EREW PRAM.

TIME AND WORK BOUNDS: The bound of $O(\log n)$ on the time is trivial. For the bound on the work, it is easy to see that it is $O(n)$ until step 5. So, we only need to bound the work of the two nested for cycles. Let n_c be the number of nodes with color $c > 2$ after step 5 has been executed. The work for recoloring all nodes of color c is $O(n_c)$. Therefore, $\sum_c O(n_c) = O(\sum_c n_c) = O(n)$.